ECE 414/517: Reinforcement Learning

Homework 2 + Coding Assignment 2

Fall 2026

Instructor: Fei Liu

Released: Thursday, October 15, 2026
Due: Wednesday, November 4, 2026 at 11:59 p.m. ET

Purpose and learning objectives

This assignment develops model-free prediction and control from sampled experience. After completing it, you should be able to (i) compare Monte Carlo, TD(0), and n-step targets; (ii) distinguish SARSA, Expected SARSA, and Q-learning; (iii) reason about exploration, state discretization, and episode endings; and (iv) implement and evaluate a tabular Q-learning agent for CartPole. No neural networks are required.

Submission instructions

Submit the following two items separately on Canvas.

1. **Homework 2 (written):** one PDF containing answers to Problems 1–5. Use LaTeX, Word, or very neat handwriting. Show intermediate calculations and explain your reasoning; unreadable work may lose points.

2. **Coding Assignment 2:** one ZIP containing your completed `coding/` directory and a `report.pdf` containing the coding experiments and interpretation. Include the small CSV result files used for your plots. Exclude virtual environments, caches, and saved Q-tables.

Use Python 3.10 or newer, NumPy, Gymnasium, and Matplotlib. Install dependencies from `coding/requirements.txt`. Follow the course collaboration, late-work, and academic-integrity policies posted on Canvas. Written and coding portions are each graded out of 100 points. The ECE 517 extension is included within the coding rubric.

Conventions

The reward R_{t+1} is received after action A_t. Terminal states have value zero. Unless stated otherwise, numerical questions use the values given at the start of that question; do not carry updated values between independent subparts. Round numerical answers to four decimal places when needed. No formal convergence proof is required.

Homework 2: Written Problems (100 points)

1 Monte Carlo, TD(0), and n-step prediction (25 points)

A fixed policy generates the episode

$$A \xrightarrow{R_1=1} B \xrightarrow{R_2=-1} C \xrightarrow{R_3=3} T,$$

where T is terminal. Let $\gamma = 0.9$, $\alpha = 0.5$, and initially $V(A) = 2$, $V(B) = 1$, $V(C) = 4$, $V(T) = 0$.

(a) Compute the full Monte Carlo returns G_0, G_1, G_2. [6 pts]

(b) Starting from the initial values, process the episode once using online TD(0). Show each target, TD error, and updated state value. [6 pts]

(c) Using the *initial* values for bootstrapping, compute the two-step target from A and the corresponding update to $V(A)$. Compute the two-step target from B, accounting for termination. [5 pts]

(d) Starting again from the initial values, perform one constant-step-size Monte Carlo update to each visited nonterminal state. Compare the new $V(A)$ with part (b). [4 pts]

(e) Explain how increasing n changes reliance on bootstrapping versus sampled rewards. Discuss the usual bias–variance tradeoff without claiming that either method must outperform the other on every task. [4 pts]

2 SARSA, Expected SARSA, and Q-learning (25 points)

Background: Expected SARSA (provided for this question). No prior coverage of Expected SARSA is assumed. Like SARSA, it learns action values from sampled transitions. The difference is how it handles the next action: SARSA uses one sampled next action, whereas Expected SARSA uses a **probability-weighted average over all next actions**.

For a nonterminal transition (s, a, r, s'), let $\pi(b \mid s')$ be the probability of choosing action b in the next state under the specified policy. First compute

$$\overline{Q}_\pi(s') = \sum_b \pi(b \mid s') Q(s', b).$$

Then compute the target, TD error, and update in that order:

$$y_{\mathrm{ES}} = r + \gamma \overline{Q}_\pi(s'), \qquad \delta_{\mathrm{ES}} = y_{\mathrm{ES}} - Q(s, a), \qquad Q(s, a) \leftarrow Q(s, a) + \alpha \delta_{\mathrm{ES}}.$$

Here b runs over all available actions; it is a summation index, not the particular action observed in the trajectory. For two actions L, R, the average is

$$\overline{Q}_\pi(s') = \pi(L \mid s') Q(s', L) + \pi(R \mid s') Q(s', R).$$

Use the ϵ-greedy policy specified below. With m actions and a *unique* greedy action b^*, uniform exploration over all actions gives

$$\pi(b \mid s') = \begin{cases} 1 - \epsilon + \epsilon/m, & b = b^*, \\ \epsilon/m, & b \neq b^*. \end{cases}$$

Thus the greedy action can be selected by either exploitation or exploration. The probabilities must sum to one; the average is not generally an unweighted arithmetic mean. For comparison,

$$y_{\text{SARSA}} = r + \gamma Q(s', A_{t+1}), \qquad y_{\text{Q-learning}} = r + \gamma \max_b Q(s', b).$$

Expected SARSA does not use the single sampled A_{t+1} in its target. It averages only over the next action at the *observed* state s'; it does not require a transition model. If s' is terminal, all three targets are simply r. In this question Expected SARSA averages under the behavior policy, so its target policy is that same ϵ-greedy policy.

Use the following data for parts (a)–(d). An agent observes $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}) = (s, a, 2, s', L)$, where s' is nonterminal. Initially $Q(s, a) = 1$, and

$$Q(s', L) = 2, \qquad Q(s', R) = 6.$$

Let $\alpha = 0.2$, $\gamma = 0.9$, and use an ϵ-greedy behavior policy with $\epsilon = 0.2$. During exploration, select uniformly from *both* actions.

(a) Compute $\pi(L \mid s')$ and $\pi(R \mid s')$. [4 pts]

(b) Independently calculate the target, TD error, and updated $Q(s, a)$ for SARSA, Expected SARSA under this policy, and Q-learning. [12 pts]

(c) Identify the target policy of each update. Explain why Q-learning is off-policy even when the agent collects its own experience using an ϵ-greedy policy. [5 pts]

(d) In a cliff-walking task, explain why SARSA can prefer a safer route while Q-learning learns a greedy route near the cliff. Distinguish reward during exploratory training from performance during greedy evaluation. [4 pts]

3 Exploration and evaluation (15 points)

Consider a tabular agent with all Q-values initially zero. Let $k = 0, 1, \ldots$ denote the episode number. Candidate exploration schedules are

$$\epsilon_k = 0, \quad \epsilon_k = 0.1, \quad \epsilon_k = (k+1)^{-1/2}, \quad \epsilon_k = \exp(-k).$$

(a) Explain how greedy action selection from the beginning can prevent useful exploration. How can deterministic tie-breaking influence early behavior? [4 pts]

(b) Which schedules become greedy in the limit? Which have divergent $\sum_{k=0}^{\infty} \epsilon_k$? Give a short justification. Explain why a divergent sum alone does *not* guarantee that every state-action pair in an arbitrary MDP is visited infinitely often. [6 pts]

(c) During evaluation we set $\epsilon = 0$ and freeze the Q-table. Explain both choices and why evaluation should use fresh episodes. Can one high-scoring training episode establish reliable performance? [5 pts]

4 Discretizing CartPole (20 points)

Remark: This problem is a simplified version of the discretization task in the coding assignment; you are welcome to work on both simultaneously to connect the hand calculations with your implementation and check your understanding of bin mapping.

The continuous observation is (x, v, θ, ω): cart position, cart velocity, pole angle, and angular velocity. For this written problem, each component has *four bins* defined by three internal thresholds:

Component	Internal thresholds
x	$-1.2,\ 0,\ 1.2$
v	$-1,\ 0,\ 1$
θ	$-0.1,\ 0,\ 0.1$
ω	$-1,\ 0,\ 1$

Bin indices are $0, 1, 2, 3$. Equality belongs to the bin to the right; the outer bins extend to infinity. For example, $x = 0$ belongs to bin 2.

(a) Discretize $(0.2, -0.4, 0.05, 1.5)$ and $(-1.3, 0, -0.12, -2)$. Show all four indices for each observation. [6 pts]

(b) With two actions, how many scalar Q-values are stored with four bins per component? With 25 bins per component? How much memory in MiB does the latter require using 64-bit floats (1 MiB $= 2^{20}$ bytes)? [6 pts]

(c) Explain state aliasing and why two observations in the same bin may require different actions. What is the tradeoff between coarse and fine binning? [5 pts]

(d) Does discretizing observations guarantee that the resulting process is Markov? Explain briefly. [3 pts]

5 Episode endings and reward accounting (15 points)

Let $Q(s, a) = 4$, $\alpha = 0.1$, $\gamma = 0.95$, and $\max_{a'} Q(s', a') = 10$. The environment's reward is $+1$ on the transition.

(a) If the pole falls and the transition is genuinely terminal, calculate the Q-learning target and updated value using the environment reward. [4 pts]

(b) If the episode is cut off only by a time limit, treat that limit as an external sampling cutoff for an underlying task that continues until failure. Calculate the target and update. Explain why bootstrapping is appropriate under this convention. [4 pts]

(c) The supplied starter replaces the reward on a failure transition by -475. Recompute part (a) using that training reward. Explain why the number of balanced steps and the shaped training return are now different metrics, and why this reward change is not automatically policy-invariant. [4 pts]

(d) If a 500-step horizon were instead an intrinsic part of the task, how should the state representation and the end-of-horizon target change? [3 pts]

Coding Assignment 2: Tabular Q-learning (100 points)

CartPole task

Control a cart by selecting action 0 (push left) or 1 (push right). The observation is (x, v, θ, ω). An episode ends on physical failure or at the 500-step time limit. A score is the number of environment steps in the episode, including the final transition, and is at most 500. The learning algorithm uses a discrete Q-table, not a neural network.

The provided main program, `coding/cartpole.py`, applies a failure penalty: on genuine termination, replace the environment reward by -475; otherwise use the returned reward. Use **episode length** for all required score plots and comparisons, not the sum of shaped rewards. The CSV records both so that you can distinguish them.

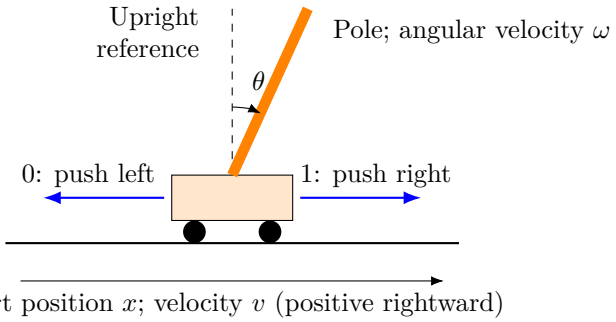


Figure 1: Push the cart to keep the pole upright. The agent observes (x, v, θ, ω) and selects one of two actions. Schematic, not to scale.

What is a bin? Why do we need one?

The environment returns four real-valued measurements. A finite Q-table cannot store a different entry for every possible real number, so we group nearby measurements into intervals called **bins**. Each bin receives an integer index. This conversion is called **discretization**. For example, four bins with boundaries $-1, 0, 1$ are

$$\underbrace{(-\infty, -1)}_{0}, \quad \underbrace{[-1, 0)}_{1}, \quad \underbrace{[0, 1)}_{2}, \quad \underbrace{[1, \infty)}_{3}.$$

Thus -0.4 maps to index 1, 0.2 maps to index 2, and exactly 0 maps to index 2: equality belongs to the interval on the right. Four bins are used here only to illustrate the idea; the required experiments use 25 per component.

Component	Meaning (units)	Discretization design range
x	Cart position (m)	$[-4.8, 4.8]$
v	Cart velocity (m/s)	$[-5, 5]$
θ	Pole angle from upright (rad)	$[-0.418, 0.418]$
ω	Pole angular velocity (rad/s)	$[-5, 5]$

These ranges determine where bin boundaries are placed. They are sometimes called clipping ranges, but **you do not modify the physical state or end an episode at these endpoints**. Out-of-range values map to the nearest edge bin. The environment separately determines failure and supplies the episode-ending flags.

Using cart position x as an example, the range $[-4.8, 4.8]$ is represented by 25 bins with the following 24 internal boundaries (in meters):

$$\mathbf{b}_x = [-4.416,\ -4.032,\ -3.648,\ \ldots,\ 3.648,\ 4.032,\ 4.416].$$

The dots indicate the omitted, equally spaced boundaries. Values below -4.416 belong to bin 0, values in $[-4.416, -4.032)$ belong to bin 1, and values at or above 4.416 belong to bin 24. The first and last bins extend outward without limit. Apply this mapping separately to all four measurements to obtain $(i_x, i_v, i_\theta, i_\omega)$. The table entry `Q[ix,iv,itheta,iomega,action]` estimates the discounted return for that state-action pair. Many continuous observations can share one table entry.

How to read the supplied code

File guide and implementation requirements

The student ZIP contains a `coding/` folder. Implement the five TODO functions in `q_learning.py`, preserving their names and arguments. The rules below are **requirements for those functions**, not additional experiments. The interaction loop is already supplied.

- `coding/q_learning.py` contains five student functions. Complete the marked TODO blocks. The function-by-function guide below explains the inputs, outputs, and behavior expected of each one.
- `coding/cartpole.py` is the provided main program. It supplies interaction, seeding, epsilon schedules, CSV output, and greedy evaluation. It creates the environment and table, selects the exploration probability for each episode, calls your functions to act and learn, and stops on termination or truncation. After training it evaluates the frozen table on 100 fresh episodes with greedy actions. You normally need not edit it.
- `coding/plot_results.py` plots non-overlapping 100-episode mean scores for the CSV files you select and saves a PNG. Supply files for one configuration across seeds in each call; do not mix exploration schedules. It adds an across-seed sample-standard-deviation band when given multiple seeds.
- `coding/test_public.py` checks boundary mapping, table shape, action selection, and updates on small examples. Passing these tests is useful but does not establish learning quality or cover every possible bug.
- `coding/requirements.txt` lists the packages for running experiments, tests, and plots. Install them using the command below. No GPU is needed.

1. **`create_discretization`: build boundaries.** Inputs are four lower bounds, four upper bounds, and the number n of bins. Return shape `(4,n-1)`. For each component use `np.linspace(low, high, n+1)[1:-1]`; do not include the endpoints as internal boundaries. The main program passes the ranges in the table and defaults to $n = 25$.

2. **`get_discretized_state`: map an observation.** Inputs are one four-number observation and the boundary array (called `buckets`). Return a tuple of four integer indices from 0 to $n - 1$. Use

right-bin equality and edge-bin saturation. The function `np.digitize(value, boundaries, right=False)` implements this rule.

3. **`create_q_table`: initialize estimates.** Given n, return a floating-point zero array of shape `(n,n,n,n,2)`. Four axes represent discretized observations; the last represents actions 0 and 1.

4. **`epsilon_greedy`: choose one action.** Inputs are the table, current discrete state, and ϵ. With probability ϵ, explore by selecting uniformly from *both* actions. Otherwise choose the action with the larger Q-value. If the values tie, choose action 0, the smaller index, as `np.argmax` does. Return an integer 0 or 1. For values $[2, 6]$ and $\epsilon = 0.2$, the probabilities are 0.1 for action 0 and 0.9 for action 1. Use `np.random` so the seed set in the main program controls sampling. The main program sets the schedule; this function only uses the supplied ϵ.

5. **`q_update`: learn from a transition.** Inputs include the current state, action, reward, next state, `learning_rate` (α), `discount` (γ), and `terminated`. Change only the selected entry *in place*:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[y - Q(s,a)], \qquad y = \begin{cases} r, & \text{if terminated,} \\ r + \gamma \max_{a'} Q(s',a'), & \text{otherwise.} \end{cases}$$

   The main program already applies the failure penalty; do not apply it again. Physical failure has `terminated=True`, so do not bootstrap. A time-limit-only cutoff has `truncated=True` but is not a true termination under this assignment's convention, so it still bootstraps.

Understanding the output files. An output path `results/linear_seed1.csv` produces that training file and `linear_seed1.evaluation.csv`. Each training row is one episode: `score` is episode length, `training_return` is the sum of shaped rewards, and `epsilon` is exploration probability; other columns record configuration. The evaluation file records fresh greedy scores. These files support reproducibility; **your report must present plots and a summary table**, rather than asking the grader to read raw CSVs.

Setup and quick execution check

From the package's root directory, install dependencies. After completing the five functions, run the tests and a short trial:

```
python -m pip install -r coding/requirements.txt
python -m pytest coding/test_public.py
python coding/cartpole.py --episodes 10 --eval-episodes 5 \
  --seed 1 --output results/quick_check.csv
```

Tests are expected to fail until the TODO blocks are implemented. This quick execution check (sometimes called a smoke test) verifies that the program runs and writes both output files without errors; it does not establish learning quality. Use a new output filename each time: the main program refuses to overwrite results. Do not use this short trial for your required learning curves.

Tasks and grading

1. **Discretization and Q-table (25 points). File to edit: `coding/q_learning.py`.** Implement the three functions below, respecting the boundary and shape conventions:
   - `create_discretization` [10 pts];
   - `get_discretized_state` [10 pts];
   - `create_q_table` [5 pts].

2. **Action selection and learning (30 points). File to edit: `coding/q_learning.py`.** Implement `epsilon_greedy` [10] and `q_update` [20]. The latter updates the table in place and must handle genuine terminal transitions correctly.

3. **Random-action baseline (10 points). Run: `coding/cartpole.py`; report in `report.pdf`.** Run 10,000 episodes with constant $\epsilon = 1$, seed 1, $\alpha = 0.1$, and $\gamma = 0.95$. Report the mean episode length over the final 1,000 episodes. Explain the action-selection policy and why Q-updates alone cannot improve its behavior while ϵ remains one. Save the generated training and evaluation CSV files under `results/`.

4. **Exploration experiments (20 points). Run: `coding/cartpole.py`; plot with `coding/plot_results.py`; present results in `report.pdf`.** Compare constant $\epsilon = 0.1$ with the supplied linear decay

$$\epsilon_k = \max\left(0.01,\ 1 - (1 - 0.01)\frac{k}{8000}\right), \quad k = 0, \ldots, 9999.$$

   Run each setting for 10,000 episodes using seeds 1, 2, and 3, keeping other parameters fixed. Plot the 100-episode block mean scores, averaged across seeds; show a clearly labeled standard-deviation band. Report each seed's final 1,000-episode training mean and its mean score over 100 fresh greedy evaluation episodes. Compare the aggregate results and explain the training/evaluation gap. You may additionally tune α, γ, or the epsilon schedule; document every final configuration and label additional experiments separately. Keep the generated CSV files and plots in `results/`. The plotting helper produces training curves; prepare the evaluation figure and summary table using your own analysis script (for example, `coding/analyze_results.py`, which is not supplied) or another tool.

5. **Interpretation and reproducibility (15 points). Write in: `report.pdf`; supporting files: `coding/` and `results/`.** Explain the effect of discretization and exploration, discuss variability across runs, and report limitations or failed runs honestly. Aim for a final 1,000-episode training mean of at least 195, but credit depends on correct implementation, controlled experiments, and sound analysis rather than a single stochastic threshold. Include exact commands, seeds, dependency versions, plots, and source CSVs in your report/package.

ECE 517 graduate extension. Run `coding/cartpole.py` with `-bins 10`, save outputs in `results/`, and include the comparison in `report.pdf`. Repeat the linear-decay experiment with 10 bins per component for seeds 1, 2, and 3. Compare Q-table size, final training performance, and greedy evaluation against the 25-bin results. Explain the generalization/data-coverage tradeoff. This extension is assessed within the 15 points for interpretation and reproducibility.

Coding rubric

Component	Points
Discretization and Q-table	25
Epsilon-greedy and Q-update	30
Random-action baseline	10
Exploration experiments	20
Interpretation and reproducibility (including ECE 517 extension)	15
Total	100

Report and submission workflow

What is supplied? The package provides the unfinished learning functions in `coding/q_learning.py`, the main program `coding/cartpole.py`, the training-curve helper `coding/plot_results.py`, tests, and dependencies. It does **not** include completed experiment results, an evaluation-plot/table helper, or `report.pdf`. You must create these outputs. Run all commands from the package root, which contains `coding/`.

Step 1: implement and test

Complete the five TODO functions in `coding/q_learning.py`, then run:

```
python -m pip install -r coding/requirements.txt
python -m pytest coding/test_public.py
```

Step 2: generate training and evaluation data

Run the supplied `coding/cartpole.py` using the commands below. Each run defaults to 10,000 training episodes, 100 greedy evaluation episodes, 25 bins, $\alpha = 0.1$, and $\gamma = 0.95$. It creates the named training CSV plus a matching evaluation CSV, for example `random_seed1.evaluation.csv`. Existing output files are not overwritten; use new paths when rerunning.

How to read the training CSV. The first row contains column names. Each subsequent row describes **one completed training episode**, not one environment step. A full run contains 10,000 data rows. The columns are:

Column	Meaning
`episode`	Training episode number, starting at 1 and ending at 10,000.
`score`	Number of environment steps in this episode, including its final transition (at most 500). Use this column for learning curves and performance means.
`training_return`	Sum of the rewards actually used for learning in this episode, without discounting this logged sum. A failure transition contributes -475 instead of $+1$, so this is not the episode-length score.
`epsilon`	Exploration probability used throughout this episode. It is constant or follows the selected linear schedule.
`seed`	Training-run seed supplied with `-seed`; the same value appears in every row of this run.
`bins`	Number of bins per observation component: normally 25, or 10 for the graduate comparison.
`alpha`	Q-learning step size, supplied with `-alpha`; default 0.1.
`gamma`	Discount factor, supplied with `-gamma`; default 0.95.
`schedule`	Exploration schedule name: `constant` or `linear`.

For example, if an episode lasts 20 steps and ends in physical failure, its `score` is 20 and its `training_return` is $19 - 475 = -456$. For a 500-step time-limit-only ending, both values are 500. The logged sum is undiscounted even though the Q-learning update uses `gamma`.

How to read the evaluation CSV. The matching `.evaluation.csv` also begins with a header. Each of its 100 data rows describes **one fresh greedy evaluation episode**, run after training with the Q-table frozen (no learning updates and no exploratory actions).

Column	Meaning
`episode`	Evaluation episode number, starting again at 1 and ending at 100.
`score`	Episode length, including the final transition (at most 500). Average these 100 values to obtain this trained agent's evaluation mean.
`seed`	Environment-reset seed for this particular evaluation episode, not the training-run seed. It equals training seed $+100000 +$ `episode` $- 1$.

For a run trained with seed 1, evaluation seeds are 100001 through 100100. All 100 evaluation rows assess the *same* learned Q-table. Identify the training run by the matching filename and its training CSV, not by treating evaluation seeds as additional training runs. The evaluation CSV has no `training_return` column because the required evaluation metric is episode length.

Random baseline.

```
python coding/cartpole.py --schedule constant --epsilon 1 \
  --seed 1 --output results/random_seed1.csv
```

Constant exploration.

```
python coding/cartpole.py --schedule constant --epsilon 0.1 \
  --seed 1 --output results/constant_seed1.csv
```

```
python coding/cartpole.py --schedule constant --epsilon 0.1 \
  --seed 2 --output results/constant_seed2.csv
python coding/cartpole.py --schedule constant --epsilon 0.1 \
  --seed 3 --output results/constant_seed3.csv
```

Linear decay.

```
python coding/cartpole.py --schedule linear \
  --seed 1 --output results/linear_seed1.csv
python coding/cartpole.py --schedule linear \
  --seed 2 --output results/linear_seed2.csv
python coding/cartpole.py --schedule linear \
  --seed 3 --output results/linear_seed3.csv
```

Step 3: generate training-curve figures

Use the supplied `coding/plot_results.py` after the runs finish:

```
python coding/plot_results.py results/random_seed1.csv \
  --output results/random_training.png
python coding/plot_results.py results/constant_seed1.csv \
  results/constant_seed2.csv results/constant_seed3.csv \
  --output results/constant_training.png
python coding/plot_results.py results/linear_seed1.csv \
  results/linear_seed2.csv results/linear_seed3.csv \
  --output results/linear_training.png
```

Embed these three PNGs in your report. The helper produces 100-episode block means and across-seed sample-SD bands for multi-seed runs. It only makes training curves; do not pass evaluation CSVs to it. If using separate panels, adjust the helper's vertical-axis limits as needed to make the panels comparable.

Step 4: create the evaluation figure and summary table

No supplied helper performs this step. Use your own Python script or a spreadsheet. For each of the seven runs:

1. Read `score` in the training CSV and average its last 1,000 rows.

2. Read `score` in the matching `.evaluation.csv` and average all 100 rows. This is one evaluation mean for that training seed.

3. Plot the evaluation means as points grouped by Random, Constant, and Linear, with seed labels and a group mean. Random has one point; the others have three. Save, for example, `results/greedy_evaluation.png`.

4. Create a table with configuration, bins, seed, final 1,000-episode training mean, and 100-episode evaluation mean. Add mean ± sample SD aggregate rows for Constant and Linear, computed across three per-seed means for each metric. Use `np.std(values, ddof=1)` or an equivalent sample-SD calculation. Do not report an across-seed SD for the one-seed Random baseline.

Do not use `training_return` as the score. Embed the evaluation figure and formatted table in your report. If you create your own script named `coding/analyze_results.py`, run it with `python coding/analyze_results.py` after implementing it; this file is **not supplied**. Include your script or analysis workbook in the ZIP.

Step 5: ECE 517 additional runs

Run the main program and training-curve helper for 10 bins:

```
python coding/cartpole.py --schedule linear --bins 10 \
  --seed 1 --output results/linear10_seed1.csv
python coding/cartpole.py --schedule linear --bins 10 \
  --seed 2 --output results/linear10_seed2.csv
python coding/cartpole.py --schedule linear --bins 10 \
  --seed 3 --output results/linear10_seed3.csv
python coding/plot_results.py results/linear10_seed1.csv \
  results/linear10_seed2.csv results/linear10_seed3.csv \
  --output results/linear10_training.png
```

Include the extra training plot. Extend your evaluation figure and summary table from step 4 with the three 10-bin runs and their aggregate. Compare these with the 25-bin linear results, including Q-table size and the data-coverage tradeoff.

Step 6: assemble your report and submit

You create `report.pdf`; no supplied program generates it. Use LaTeX, Word, or another document editor to combine the figures, table, and your explanations. Export to PDF. Include exact commands, seeds, settings, dependency versions, limitations, and explanations of variability and the training/evaluation gap.

- Written PDF: Problems 1–5 with calculations and explanations (100 points).
- Coding ZIP: `coding/`, `report.pdf`, and `results/` with the CSVs and plots used in the report. No Q-tables or virtual environments.
- Include training curves, a greedy-evaluation figure, and the numerical summary table. CSV-only submissions are insufficient. Separate training scores from greedy evaluation scores.
- Verify public tests pass and confirm your ZIP contains the actual completed source files. Public tests are illustrative and are not the entire grading rubric.
- ECE 517: include the 10-bin comparison within the coding report.